

Roteiro Prático

1. Fundamentação e Configuração do Ambiente

Na moderna Gerência de Configuração (GC), o Git não é apenas uma ferramenta de versionamento, mas o alicerce da integridade e evolução do software. Ele atua como uma rede de segurança contra a "Síndrome do Arquivo Final" — o caos gerado por nomenclaturas como `projeto_final_v2_agora_vai.zip` que imobiliza equipes pelo medo de quebras irreparáveis. Como especialistas, entendemos que o Git transforma o desenvolvimento em um processo auditável, onde cada alteração é um passo documentado na evolução do código.

O marco zero de qualquer projeto profissional é o comando:

```
git init
```

Ao executar este comando, o Git cria o diretório oculto `.git` (o Repositório), estabelecendo a separação fundamental entre o seu Working Directory (onde os arquivos são editados), a Staging Area (onde as mudanças são preparadas) e o histórico permanente.

Identidade Estratégica vs. Autenticação

Configurar a identidade é o primeiro passo para a rastreabilidade corporativa:

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seu.email@empresa.com"
```

Análise "So What?": É vital distinguir: o `git config` define quem assina o commit (Autor/Committer) para fins de auditoria, mas não autentica o desenvolvedor no servidor (GitHub/GitLab). A autenticação real é feita via chaves SSH ou Personal Access Tokens (PAT), garantindo que, embora qualquer um possa "assinar" um commit localmente, apenas usuários autorizados possam enviá-lo ao servidor.

Blindagem e Higiene: .gitignore

Para evitar o inchaço do repositório com arquivos irrelevantes como `node_modules/`, logs ou arquivos de sistema (`.DS_Store`), utilizamos o arquivo `.gitignore`.

Arquivos listados no `.gitignore` não são rastreados pelo Git mesmo quando modificados, mantendo o repositório limpo e focado em código produtivo.

2. Construção do MVP e Snapshot Inicial

Um histórico profissional exige commits granulares. Registrar marcos estáveis permite que a equipe entenda a evolução lógica do produto sem enfrentar blocos de código massivos e indecifráveis.

Estrutura do Projeto Kanban (Código Base)

Criaremos a base do nosso sistema:

- `index.html`: Estrutura de colunas (To Do, Doing, Done).
- `style.css`: Estilização minimalista.
- `app.js`: Lógica de inicialização.

Criem código para esses arquivos, podem usar IA para ajudar a criar o código. O importante é o uso do Git.

Ciclo de Trabalho Profissional

Mova os arquivos para a *Staging Area* e realize o primeiro snapshot:

```
git add .
```

```
git commit -m "feat: estrutura inicial do kanban corporativo"
```

3. Desenvolvimento Paralelo: A Branch de Funcionalidade

Branches são ponteiros móveis que permitem o isolamento completo de tarefas. Elas possibilitam o trabalho assíncrono, garantindo que a branch principal (`main`) permaneça sempre estável (produtiva).

Trabalho em Feature

Adotaremos padrões de nomenclatura maduros (Naming Conventions), incluindo o ID da tarefa para integração com ferramentas de gestão:

```
git switch -c feat/JIRA-101-filtro-prioridade
```

Após adicionar a lógica de filtro no `app.js`, registramos a evolução:

```
git add app.js
```

```
git commit -m "feat(JIRA-101): adiciona lógica base para filtro de prioridade"
```

O uso do comando moderno `git switch` em vez do antigo `checkout` torna a intenção do desenvolvedor mais clara e semântica. Nomes como `feat/` ou `bugfix/` aliados a IDs de ticket garantem escalabilidade e transparência no fluxo de trabalho de grandes equipes.

4. Gerenciamento de Crise: O Fluxo de Hotfix

Imagine uma emergência em plena "Sexta-Feira de Black Friday": o botão de salvar tarefas parou de funcionar. Você não pode esperar a conclusão da feature de filtros para corrigir a produção.

Resposta à Crise

1. Retorno ao Estado Estável: `git switch main`
2. Isolamento do Erro: `git switch -c hotfix/validacao-critica`
3. Correção e Registro: Aplique o fix e comite.

```
git add app.js
```

```
git commit -m "fix: corrige validação de entrada de dados para evitar crash"
```

O hotfix isolado permite que a correção seja integrada e enviada para produção imediatamente. Tentar corrigir bugs diretamente em branches de desenvolvimento "poluídas" atrasa a continuidade do negócio e introduz riscos desnecessários.

5. Refinamento de Histórico e Recuperação Moderna

A qualidade do log de commits reflete a senioridade do engenheiro. O Git permite ajustar o histórico local antes que ele se torne público.

O Poder do Amend

Esqueceu de incluir um arquivo `config.json` ou errou a mensagem do último commit? Faça a inclusão agora e conserte com o amend.

```
git add config.json
```

```
git commit --amend -m "fix: validação de entrada e adição de config global"
```

Atenção: O `--amend` deve ser usado apenas em commits locais. Alterar commits remotos exige um `force push`, o que pode quebrar o histórico de outros desenvolvedores.

Recuperação e Descarte (Modern Git)

Se você adicionou um arquivo por engano ou quebrou o código no Working Directory, utilize as recomendações modernas do Git:

- Tirar da Staging Area (sem perder alterações): `git restore --staged arquivo_indesejado.txt` (Substitui o antigo `reset HEAD`).
- Descartar mudanças locais e restaurar o arquivo: `git restore app.js` (Substitui o antigo `checkout --`). Este comando é destrutivo; use-o apenas quando desejar voltar exatamente ao estado do último commit.

6. Finalização e Entrega: Tags e Deploy

Tags funcionam como marcos, sinalizando versões de lançamento (releases) para as equipes de QA e Operações.

Versionamento e Push

Crie uma tag anotada para a entrega oficial:

```
git tag -a v1.0 -m "Release v1.0 - Sistema Kanban Operacional"
```

Sincronize seu trabalho com o servidor remoto:

```
git push -u origin main
```

```
git push --tags
```

A tríade de commits organizados, branches nomeadas e tags de versão transforma arquivos soltos em um produto de software profissional, auditável e pronto para o mercado.

Resumo de Comandos e Funções Estratégicas

Comando	Função Técnica	Valor Estratégico	Recomendação Moderna
<code>git init</code>	Inicializa o repositório local	Cria o "Marco Zero" e a estrutura <code>.git</code> .	Padrão
<code>git config</code>	Define Identidade (Nome/Email)	Garante rastreabilidade em auditorias.	Padrão
<code>git switch -c</code>	Cria e alterna de branch	Isolamento de riscos e trabalho paralelo.	Substitui <code>checkout -b</code>
<code>git restore</code>	Descarta mudanças no Working Dir	Recuperação rápida de código funcional.	Substitui <code>checkout --</code>
<code>git restore --staged</code>	Remove arquivos da Staging Area	"Rebaixa" o arquivo para revisão.	Substitui <code>reset HEAD</code>
<code>git commit --amend</code>	Edita o último commit local	Mantém o histórico limpo e coeso.	Usar apenas em commits locais

<code>git tag -a</code>	Cria tag anotada de versão	Define pontos de release imutáveis.	Padrão
<code>git push -u</code>	Sincroniza e vincula ao remoto	Estabelece o elo entre local e servidor.	Padrão